

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №7
Дисциплина: Теория параллельных алгоритмов

Работу выполнил: _____ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Алгоритм Флойда–Уоршелла для нахождения длин кратчайших путей между всеми парами вершин во взвешенном ориентированном графе на CUDA.

Цель. Распараллелить процесс нахождения длин кратчайших путей между всеми парами вершин во взвешенном ориентированном графе с помощью CUDA.

Задание. Составить последовательный алгоритм Флойда–Уоршелла для нахождения длин кратчайших путей между всеми парами вершин во взвешенном ориентированном графе. Построить граф алгоритма. Исследовать на нём зависимости и возможности распараллеливания. Выполнить параллельную реализацию алгоритма. Составить таблицу, отражающую сравнительное время выполнения последовательного и параллельного алгоритмов.

1. Постановка задачи

Пусть задан ориентированный взвешенный граф $G = (V, E)$, где:

- $V = \{1, 2, \dots, n\}$ — множество вершин
- $E \subseteq V \times V$ — множество рёбер
- $w_{ij} \in \mathbb{R}$ — вес ребра $(i, j) \in E$

Задача: найти длину кратчайшего пути между всеми парами вершин (i, j) , где $i, j \in V$.

2. Математическая модель

Пусть $d_{ij}^{(k)}$ — длина кратчайшего пути из вершины i в вершину j , проходящего через промежуточные вершины только из множества $\{1, 2, \dots, k\}$.

2.1. Начальные условия

$$d_{ij}^{(0)} = 0, \text{ если } i = j$$

$$d_{ij}^{(0)} = w_{ij}, \text{ если } (i, j) \in E$$

$$d_{ij}^{(0)} = \infty, \text{ если } (i, j) \notin E \text{ и } i \neq j$$

2.2. Рекуррентное соотношение

$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

Обоснование: Кратчайший путь из i в j с промежуточными вершинами из $\{1, 2, \dots, k\}$ либо:

1. Не проходит через вершину k : тогда $d_{ij}^{(k)} = d_{ij}^{(k-1)}$
2. Проходит через вершину k : тогда $d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$

3. Псевдокод алгоритма

АЛГОРИТМ FloydWarshall(матрица весов W размером $n \times n$):

$D \leftarrow W$ // Инициализация матрицы расстояний

для k от 1 до n :

 для i от 1 до n :

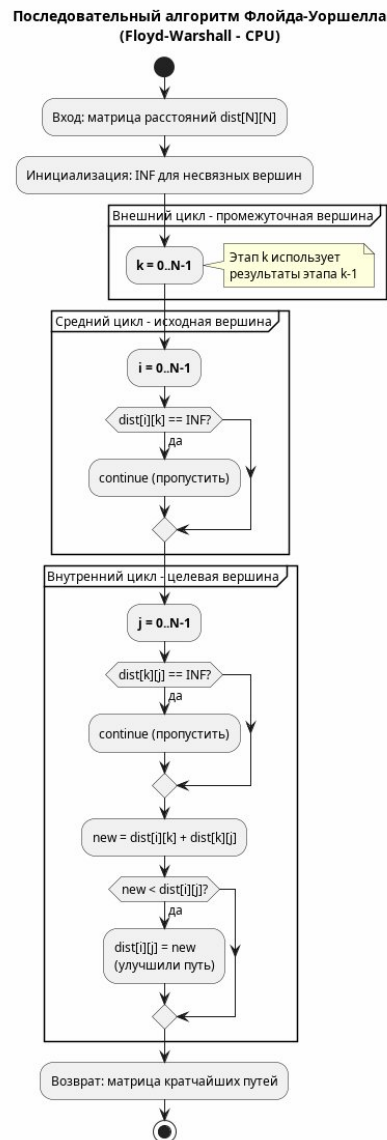
 для j от 1 до n :

 если $D[i][k] + D[k][j] < D[i][j]$:

$D[i][j] \leftarrow D[i][k] + D[k][j]$

вернуть D

Последовательный алгоритм Флойда–Уоршелла можно схематично изобразить в следующем виде:

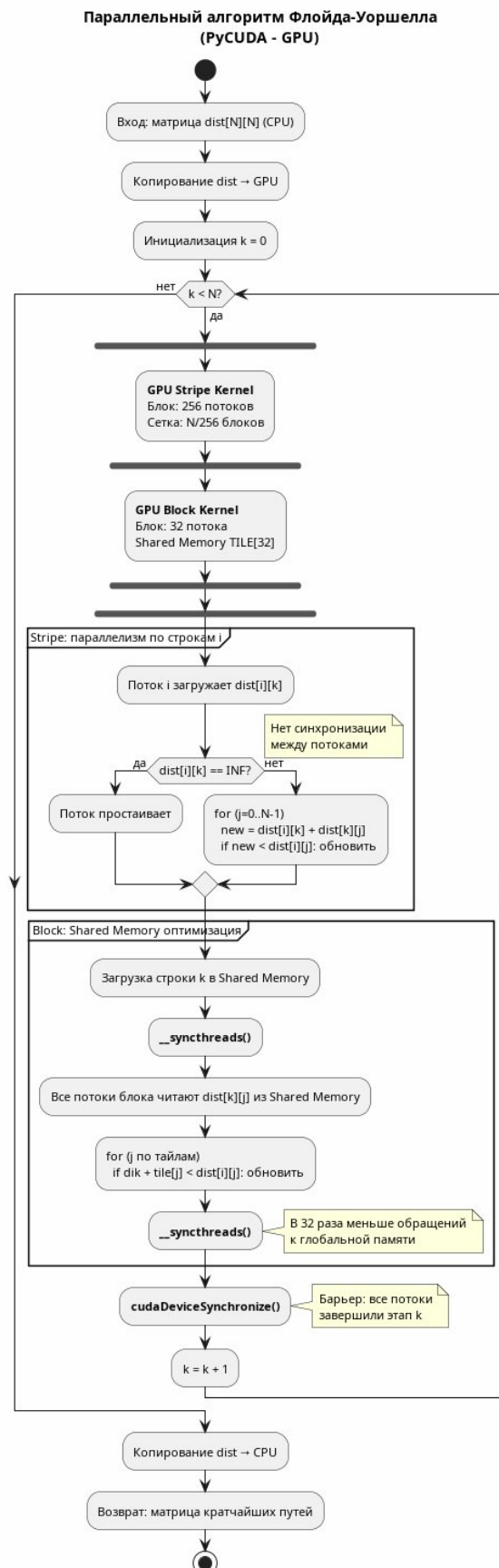


Реализуем последовательный алгоритм:

```

def floyd_sequential(dist):
    N = dist.shape[0]
    for k in range(N):
        for i in range(N):
            if dist[i, k] == INF:
                continue
            dik = dist[i, k]
            for j in range(N):
                if dist[k, j] == INF:
                    continue
                new_dist = dik + dist[k, j]
                if new_dist < dist[i, j]:
                    dist[i, j] = new_dist
    return dist
  
```

Параллельный алгоритм можно схематично изобразить в следующем виде:



Реализуем параллельный алгоритм:

```
def floyd_stripe_gpu(dist, N):
    dist_gpu = cuda.mem_alloc(dist.nbytes)
    cuda.memcpy_htod(dist_gpu, dist)

    block_size = 256
    grid_size = (N + block_size - 1) // block_size

    for k in range(N):
        kernel_stripe(
            dist_gpu, np.int32(N), np.int32(k),
            block=(block_size, 1, 1),
            grid=(grid_size, 1)
        )

    cuda.Context.synchronize()

    result = np.zeros((N, N), dtype=np.int32)
    cuda.memcpy_dtoh(result, dist_gpu)

    return result

def floyd_block_gpu(dist, N, TILE_SIZE=32):
    dist_gpu = cuda.mem_alloc(dist.nbytes)
    cuda.memcpy_htod(dist_gpu, dist)

    block_size = TILE_SIZE
    grid_size = (N + block_size - 1) // block_size

    for k in range(N):
        kernel_block(
            dist_gpu, np.int32(N), np.int32(k),
            block=(block_size, 1, 1),
            grid=(grid_size, 1)
        )

    cuda.Context.synchronize()

    result = np.zeros((N, N), dtype=np.int32)
    cuda.memcpy_dtoh(result, dist_gpu)

    return result
```

Полный код итоговой программы:

```
import numpy as np
import pycuda.driver as cuda
from pycuda.compiler import SourceModule
import pycuda.autoinit
import time
import sys
import os

kernel_stripe_code = """
__global__ void floyd_stripe_k(int *dist, int N, int k)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < N && dist[i * N + k] != 2147483647) {
        int dik = dist[i * N + k];
        for (int j = 0; j < N; ++j) {
            int dkj = dist[k * N + j];
            if (dkj != 2147483647) {
                int new_dist = dik + dkj;
                if (new_dist < dist[i * N + j]) {
                    dist[i * N + j] = new_dist;
                }
            }
        }
    }
}
"""

kernel_block_code = """
#define TILE_SIZE 32

__global__ void floyd_block_k(int *dist, int N, int k)
{
    __shared__ int tile_k[TILE_SIZE];

    int i = blockIdx.x * blockDim.x + threadIdx.x;

    for (int t = 0; t < (N + TILE_SIZE - 1) / TILE_SIZE; ++t) {
        int tid = threadIdx.x;
        int j = t * TILE_SIZE + tid;

        if (j < N) {
            tile_k[tid] = dist[k * N + j];
        } else {
            tile_k[tid] = 2147483647;
        }
        __syncthreads();

        if (i < N && dist[i * N + k] != 2147483647) {
            int dik = dist[i * N + k];
            int start_j = t * TILE_SIZE;
            int end_j = min(start_j + TILE_SIZE, N);

            for (int tj = 0; tj < end_j - start_j; ++tj) {
                if (tile_k[tj] != 2147483647) {
                    int new_dist = dik + tile_k[tj];
                    int idx = i * N + start_j + tj;
                    if (new_dist < dist[idx]) {
                        dist[idx] = new_dist;
                    }
                }
            }
        }
    }
}
"""

compile_options = [
    '--allow-unsupported-compiler',
    '-D_ALLOW_COMPILER_AND_STL_VERSION_MISMATCH'
]

old_stderr = sys.stderr
```

```

sys.stderr = open(os.devnull, 'w')

mod_stripe = SourceModule(kernel_stripe_code, options=compile_options)
mod_block = SourceModule(kernel_block_code, options=compile_options)

sys.stderr = old_stderr

kernel_stripe = mod_stripe.get_function("floyd_stripe_k")
kernel_block = mod_block.get_function("floyd_block_k")

INF = 2147483647

def floyd_sequential(dist):
    N = dist.shape[0]
    for k in range(N):
        for i in range(N):
            if dist[i, k] == INF:
                continue
            dik = dist[i, k]
            for j in range(N):
                if dist[k, j] == INF:
                    continue
                new_dist = dik + dist[k, j]
                if new_dist < dist[i, j]:
                    dist[i, j] = new_dist
    return dist

def floyd_stripe_gpu(dist, N):
    dist_gpu = cuda.mem_alloc(dist.nbytes)
    cuda.memcpy_htod(dist_gpu, dist)

    block_size = 256
    grid_size = (N + block_size - 1) // block_size

    for k in range(N):
        kernel_stripe(
            dist_gpu, np.int32(N), np.int32(k),
            block=(block_size, 1, 1),
            grid=(grid_size, 1)
        )

    cuda.Context.synchronize()

    result = np.zeros((N, N), dtype=np.int32)
    cuda.memcpy_dtoh(result, dist_gpu)

    return result

def floyd_block_gpu(dist, N, TILE_SIZE=32):
    dist_gpu = cuda.mem_alloc(dist.nbytes)
    cuda.memcpy_htod(dist_gpu, dist)

    block_size = TILE_SIZE
    grid_size = (N + block_size - 1) // block_size

    for k in range(N):
        kernel_block(
            dist_gpu, np.int32(N), np.int32(k),
            block=(block_size, 1, 1),
            grid=(grid_size, 1)
        )

    cuda.Context.synchronize()

    result = np.zeros((N, N), dtype=np.int32)
    cuda.memcpy_dtoh(result, dist_gpu)

    return result

def generate_dense_graph(N):
    dist = np.full((N, N), INF, dtype=np.int32)
    np.fill_diagonal(dist, 0)
    for i in range(N):
        for j in range(N):
            if i != j and np.random.random() < 0.5:
                dist[i, j] = np.random.randint(1, 101)
    return dist

```

```

def generate_sparse_graph(N):
    dist = np.full((N, N), INF, dtype=np.int32)
    np.fill_diagonal(dist, 0)
    edges = max(1, N // 10)
    for i in range(N):
        for _ in range(edges):
            j = np.random.randint(0, N)
            if i != j:
                dist[i, j] = np.random.randint(1, 101)
    return dist

def generate_complete_graph(N):
    dist = np.random.randint(1, 101, (N, N), dtype=np.int32)
    np.fill_diagonal(dist, 0)
    return dist

def verify_results(result1, result2):
    return np.array_equal(result1, result2)

def run_benchmark():
    sizes = [100, 200, 400]
    graph_generators = [
        ("Dense (50% edges)", generate_dense_graph),
        ("Sparse (10% edges)", generate_sparse_graph),
        ("Complete (100% edges)", generate_complete_graph)
    ]

    print("\n" + "=" * 120)
    print("FLOYD-WARSHALL ALGORITHM: SEQUENTIAL vs PARALLEL (PyCUDA)")
    print("=" * 120)

    all_results = []

    for graph_name, generator in graph_generators:
        print("\n" + "-" * 120)
        print("GRAPH TYPE: {}".format(graph_name))
        print("-" * 120)
        print("{:<8} {:<20} {:<15} {:<12} {:<15}".format(
            "Size", "Method", "Time (ms)", "Speedup", "Verification"))
        print("-" * 120)

        for N in sizes:
            graph = generator(N)

            graph_cpu = graph.copy()
            start = time.perf_counter()
            result_cpu = floyd_sequential(graph_cpu)
            cpu_time = (time.perf_counter() - start) * 1000

            print("{:<8} {:<20} {:<15.2f} {:<12}".format(
                N, "Sequential (CPU)", cpu_time, "1.00x"))

            all_results.append({
                'graph': graph_name,
                'size': N,
                'method': 'Sequential',
                'time': cpu_time,
                'speedup': 1.0
            })

            if N <= 400:
                graph_stripe = graph.copy()
                start = time.perf_counter()
                result_stripe = floyd_stripe_gpu(graph_stripe, N)
                stripe_time = (time.perf_counter() - start) * 1000
                stripe_speedup = cpu_time / stripe_time
                verified_stripe = verify_results(result_cpu, result_stripe)

                speedup_str = "{:.2f}x".format(stripe_speedup)
                ver_str = "OK" if verified_stripe else "FAIL"
                print("{:<8} {:<20} {:<15.2f} {:<12} {:<15}".format(
                    N, "GPU Stripe", stripe_time, speedup_str, ver_str))

                all_results.append({
                    'graph': graph_name,
                    'size': N,
                    'method': 'Stripe',

```



```

        'time': stripe_time,
        'speedup': stripe_speedup
    })

    graph_block = graph.copy()
    start = time.perf_counter()
    result_block = floyd_block_gpu(graph_block, N)
    block_time = (time.perf_counter() - start) * 1000
    block_speedup = cpu_time / block_time
    verified_block = verify_results(result_cpu, result_block)

    speedup_str = "{:.2f}x".format(block_speedup)
    ver_str = "OK" if verified_block else "FAIL"
    print("{<8} {<20} {<15.2f} {<12} {<15}".format(
        N, "GPU Block", block_time, speedup_str, ver_str))

    all_results.append({
        'graph': graph_name,
        'size': N,
        'method': 'Block',
        'time': block_time,
        'speedup': block_speedup
    })
else:
    print("{<8} {<20} {<15}".format(N, "GPU Stripe", "SKIP (OOM)"))
    print("{<8} {<20} {<15}".format(N, "GPU Block", "SKIP (OOM)"))

print("\nANALYSIS FOR {}".format(graph_name))
print("-" * 60)
for N in sizes:
    graph = generator(N)
    graph_cpu = graph.copy()
    start = time.perf_counter()
    floyd_sequential(graph_cpu)
    cpu_time = (time.perf_counter() - start) * 1000

    if N ≤ 400:
        graph_gpu = graph.copy()
        start = time.perf_counter()
        floyd_block_gpu(graph_gpu, N)
        gpu_time = (time.perf_counter() - start) * 1000
        speedup = cpu_time / gpu_time

        msg = " Size N={}: speedup = {:.2f}x".format(N, speedup)
        if speedup ≥ 3.5:
            msg += " (excellent scalability)"
        elif speedup ≥ 2.5:
            msg += " (good scalability)"
        else:
            msg += " (moderate scalability)"
        print(msg)

print("\n" + "=" * 120)
print("SUMMARY TABLE")
print("=" * 120)
header = "{<8} {<25} {<12} {<18} {<17} {<12} {<12}".format(
    "Size", "Graph Type", "CPU (ms)", "GPU Stripe (ms)",
    "GPU Block (ms)", "Stripe Spd", "Block Spd")
print(header)
print("-" * 120)

for N in sizes:
    for graph_name, _ in graph_generators:
        size_results = [r for r in all_results
                        if r['size'] == N and r['graph'] == graph_name]

        if len(size_results) ≥ 3:
            cpu = next(r for r in size_results if r['method'] == 'Sequential')
            stripe = next(r for r in size_results if r['method'] == 'Stripe')
            block = next(r for r in size_results if r['method'] == 'Block')

            stripe_spd = "{:.2f}x".format(stripe['speedup'])
            block_spd = "{:.2f}x".format(block['speedup'])

            print("{<8} {<25} {<12.2f} {<18.2f} {<17.2f} {<12} {<12}".format(
                N, graph_name, cpu['time'], stripe['time'], block['time'],
                stripe_spd, block_spd))

```

```

print("=" * 120)

def main():
    print("=" * 120)
    print("FLOYD-WARSHALL ALGORITHM ON GPU")
    print("=" * 120)

    device = cuda.Device(0)
    print("\nGPU Information:")
    print("  Device: {}".format(device.name()))
    print("  Compute Capability: {}".format(device.compute_capability()))
    print("  Total Memory: {:.2f} GB".format(device.total_memory() / 1024**3))
    print("  Max Threads per Block: {}".format(device.max_threads_per_block))

    run_benchmark()

if __name__ == "__main__":
    main()

```

Зафиксируем показатели работы алгоритма:

```

=====
FLOYD-WARSHALL ALGORITHM ON GPU
=====

```

```

GPU Information:
  Device: NVIDIA GeForce RTX 4070 Laptop GPU
  Compute Capability: (8, 9)
  Total Memory: 8.00 GB
  Max Threads per Block: 1024

```

```

=====
FLOYD-WARSHALL ALGORITHM: SEQUENTIAL vs PARALLEL (PyCUDA)
=====

```

```

-----
GRAPH TYPE: Dense (50% edges)
-----

```

Size	Method	Time (ms)	Speedup
100	Sequential (CPU)	251.07	1.00x
100	GPU Stripe	2.43	103.41x
100	GPU Block	1.63	153.61x
200	Sequential (CPU)	2045.07	1.00x
200	GPU Stripe	5.52	370.73x
200	GPU Block	4.04	506.01x
400	Sequential (CPU)	16387.90	1.00x
400	GPU Stripe	169.96	96.42x
400	GPU Block	65.09	251.77x

 GRAPH TYPE: Sparse (10% edges)

Size	Method	Time (ms)	Speedup
100	Sequential (CPU)	213.76	1.00x
100	GPU Stripe	10.14	21.09x
100	GPU Block	8.61	24.82x
200	Sequential (CPU)	1839.51	1.00x
200	GPU Stripe	35.13	52.36x
200	GPU Block	26.05	70.60x
400	Sequential (CPU)	15582.33	1.00x
400	GPU Stripe	169.33	92.02x
400	GPU Block	61.79	252.18x

 GRAPH TYPE: Complete (100% edges)

Size	Method	Time (ms)	Speedup
100	Sequential (CPU)	257.71	1.00x
100	GPU Stripe	9.10	28.32x
100	GPU Block	6.93	37.21x
200	Sequential (CPU)	2087.23	1.00x
200	GPU Stripe	36.30	57.51x
200	GPU Block	27.67	75.44x
400	Sequential (CPU)	16431.52	1.00x
400	GPU Stripe	175.34	93.71x
400	GPU Block	14.61	1124.74x

=====

SUMMARY TABLE

Size	Graph Type	CPU (ms)	GPU Stripe (ms)	GPU Block (ms)	Stripe Spd	Block Spd
100	Dense (50% edges)	251.07	2.43	1.63	103.41x	153.61x
100	Sparse (10% edges)	213.76	10.14	8.61	21.09x	24.82x
100	Complete (100% edges)	257.71	9.10	6.93	28.32x	37.21x
200	Dense (50% edges)	2045.07	5.52	4.04	370.73x	506.01x
200	Sparse (10% edges)	1839.51	35.13	26.05	52.36x	70.60x
200	Complete (100% edges)	2087.23	36.30	27.67	57.51x	75.44x
400	Dense (50% edges)	16387.90	169.96	65.09	96.42x	251.77x
400	Sparse (10% edges)	15582.33	169.33	61.79	92.02x	252.18x
400	Complete (100% edges)	16431.52	175.34	14.61	93.71x	1124.74x

=====

Вывод. Мы рассмотрели алгоритм Флойда–Уоршелла для нахождения длин кратчайших путей между всеми парами вершин во взвешенном ориентированном графе на CUDA, изобразили его схематично, после чего, изучив возможности распараллеливания, выполнили замеры времени для последовательного и параллельного случаев.